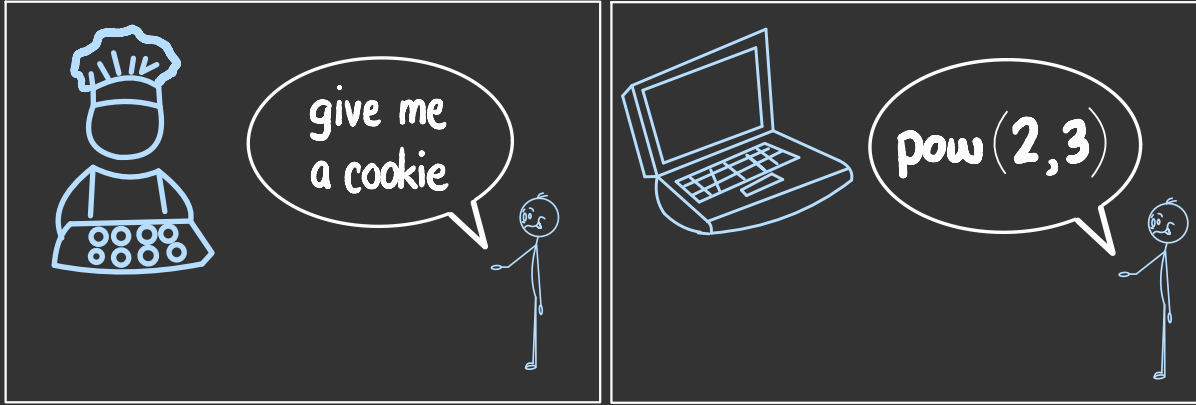


boolean expressions

CSCI
134



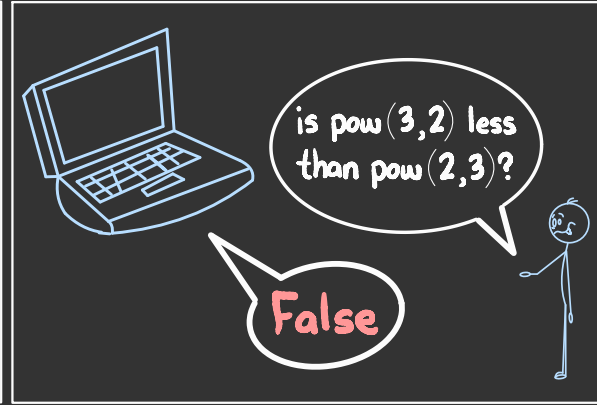
so far we've mainly used language
to issue **commands**



but language can also be used
to ask questions

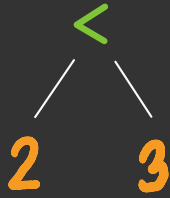


python replies to **yes/no** questions
with **True** or **False**



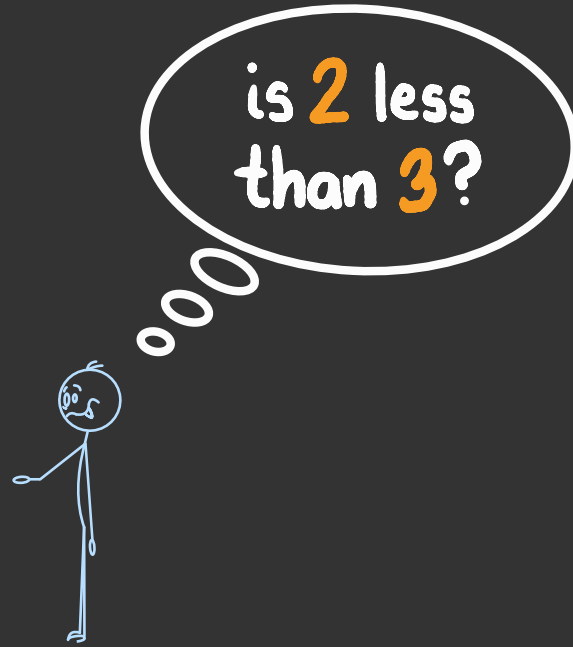
python replies to **yes/no** questions
with **True** or **False**

expression tree



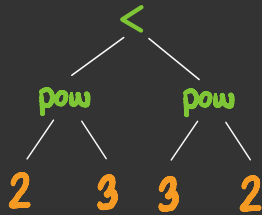
expression

2 < 3



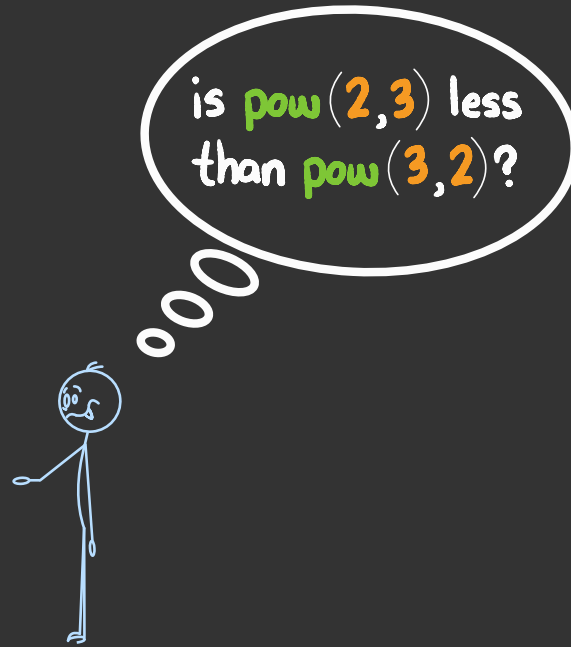
like arithmetic operators, comparison operators are flattened using the infix protocol

expression tree



expression

$\text{pow}(2,3) < \text{pow}(3,2)$



like arithmetic operators, comparison operators are flattened using the infix protocol

expression tree



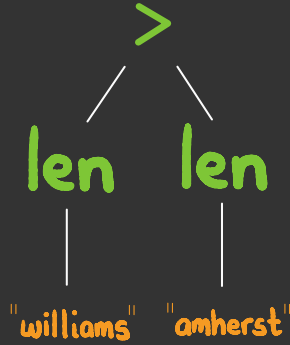
expression



is the word "williams" longer
than the word "amherst"?

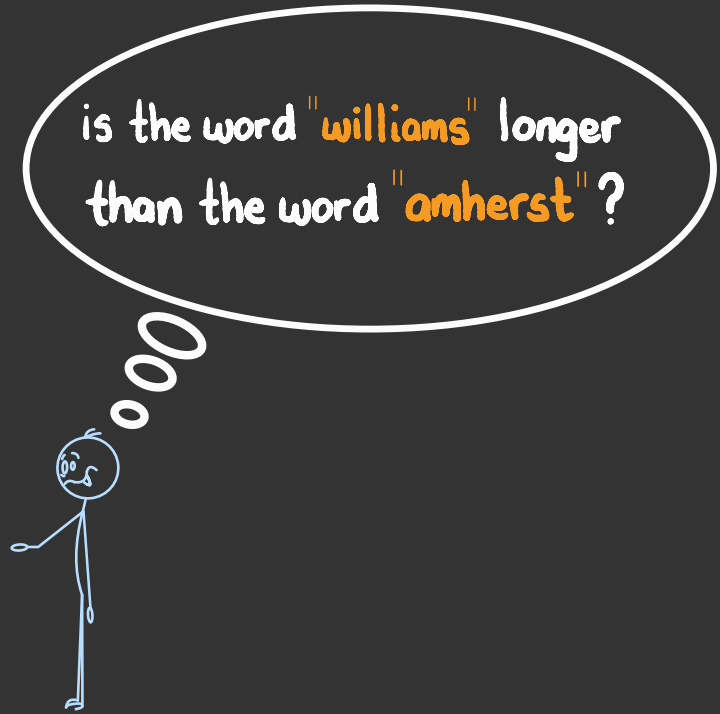


expression tree



expression

$\text{len}(\text{"williams"}) > \text{len}(\text{"amherst"})$



comparison operators

> greater than
< less than
== equal to
!= not equal to
>= greater than or equal to
<= less than or equal to

```
In [1]: 3 > 4  
Out[1]: False
```

```
In [2]: 3 < 4  
Out[2]: True
```

```
In [3]: 3 == 4  
Out[3]: False
```

```
In [4]: 3 != 4  
Out[4]: True
```

```
In [5]: 3 >= 4  
Out[5]: False
```

```
In [6]: 3 <= 4  
Out[6]: True
```

```
In [7]: 4 > 3  
Out[7]: True
```

```
In [8]: 4 < 3  
Out[8]: False
```

```
In [9]: 4 == 3  
Out[9]: False
```

```
In [10]: 4 != 3  
Out[10]: True
```

```
In [11]: 4 >= 3  
Out[11]: True
```

```
In [12]: 4 <= 3  
Out[12]: False
```

```
In [13]: 3 > 3  
Out[13]: False
```

```
In [14]: 3 < 3  
Out[14]: False
```

```
In [15]: 3 == 3  
Out[15]: True
```

```
In [16]: 3 != 3  
Out[16]: False
```

```
In [17]: 3 >= 3  
Out[17]: True
```

```
In [18]: 3 <= 3  
Out[18]: True
```

why a double equals?

== equal to

```
In [1]: 3 > 4  
Out[1]: False
```

```
In [2]: 3 < 4  
Out[2]: True
```

```
In [3]: 3 == 4  
Out[3]: False
```

```
In [4]: 3 != 4  
Out[4]: True
```

```
In [5]: 3 >= 4  
Out[5]: False
```

```
In [6]: 3 <= 4  
Out[6]: True
```

```
In [7]: 4 > 3  
Out[7]: True
```

```
In [8]: 4 < 3  
Out[8]: False
```

```
In [9]: 4 == 3  
Out[9]: False
```

```
In [10]: 4 != 3  
Out[10]: True
```

```
In [11]: 4 >= 3  
Out[11]: True
```

```
In [12]: 4 <= 3  
Out[12]: False
```

```
In [13]: 3 > 3  
Out[13]: False
```

```
In [14]: 3 < 3  
Out[14]: False
```

```
In [15]: 3 == 3  
Out[15]: True
```

```
In [16]: 3 != 3  
Out[16]: False
```

```
In [17]: 3 >= 3  
Out[17]: True
```

```
In [18]: 3 <= 3  
Out[18]: True
```

the **single** equals is already taken

the
assignment
statement

variable = expression

```
In [1]: rate = 1.35
In [2]: rate
Out[2]: 1.35

In [3]: average = (10+11+12)/3
In [4]: average
Out[4]: 11.0

In [5]: cdn = rate * 15
In [6]: cdn
Out[6]: 20.25

In [7]: injury = "boo" * 2
In [8]: injury
Out[8]: 'booboo'

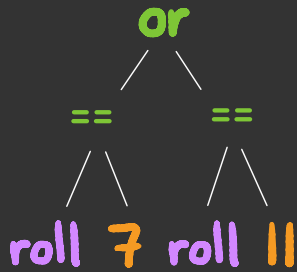
In [9]: social_security_number = "yeahright"
In [10]: social_security_number
Out[10]: 'yeahright'
```

sometimes we want to ask about
multiple things at once



for instance, a player wins a game
of craps if their first roll is 7 or 11

expression tree



expression

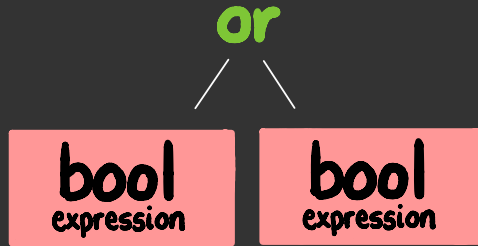
$(\text{roll} == 7) \text{ or } (\text{roll} == 11)$

is the roll equal
to 7 or 11?

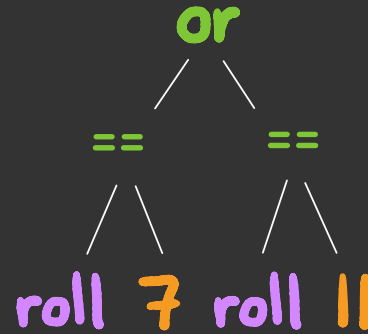


like comparison operators, "logical" operators are typically flattened using the infix protocol

expression tree



expression tree



expression



expression

$(\text{roll} == 7) \text{ or } (\text{roll} == 11)$

**bool
expression**

or

**bool
expression**

if the value of **this**
expression is **True**

or

the value of **this**
expression is **True**

then the value of the overall
expression is **True**

```
In [1]: True or True  
Out[1]: True
```

```
In [2]: (3 < 4) or (5 < 6)  
Out[2]: True
```

```
In [3]: True or False  
Out[3]: True
```

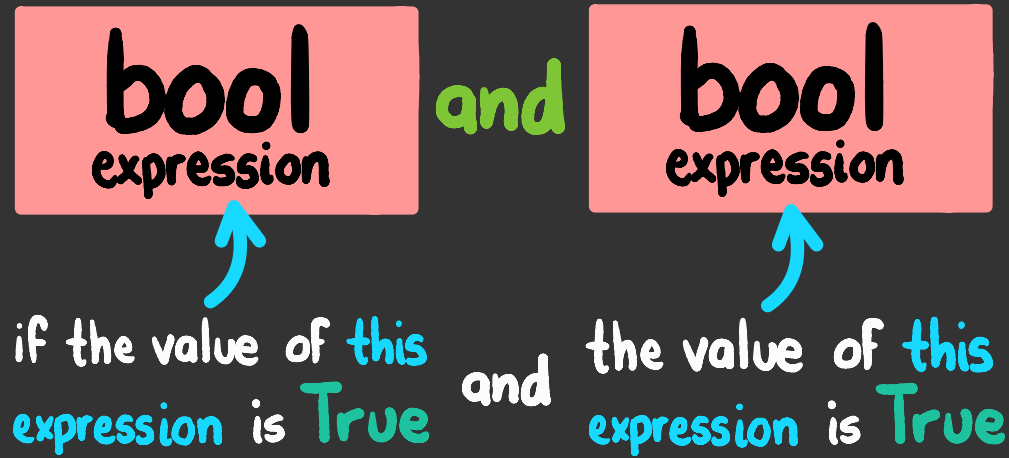
```
In [4]: (3 < 4) or (5 > 6)  
Out[4]: True
```

```
In [5]: False or True  
Out[5]: True
```

```
In [6]: (3 > 4) or (5 < 6)  
Out[6]: True
```

```
In [7]: False or False  
Out[7]: False
```

```
In [8]: (3 > 4) or (5 > 6)  
Out[8]: False
```



then the value of the overall expression is True

```
In [1]: True and True
Out[1]: True

In [2]: (3 < 4) and (5 < 6)
Out[2]: True

In [3]: True and False
Out[3]: False

In [4]: (3 < 4) and (5 > 6)
Out[4]: False

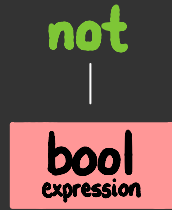
In [5]: False and True
Out[5]: False

In [6]: (3 > 4) and (5 < 6)
Out[6]: False

In [7]: False and False
Out[7]: False

In [8]: (3 > 4) and (5 > 6)
Out[8]: False
```

expression tree



expression tree



expression

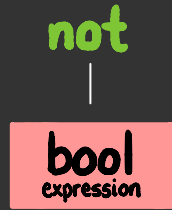
not (bool expression)

expression

not (roll == 7)

the **not** operator flips the value of a **bool** expression

expression tree



expression tree



expression

not (bool
expression)

expression

not (roll == 7)

unlike the other logical operators,
it is flattened using the prefix protocol

not

(bool
expression)



if the value of this
expression is True

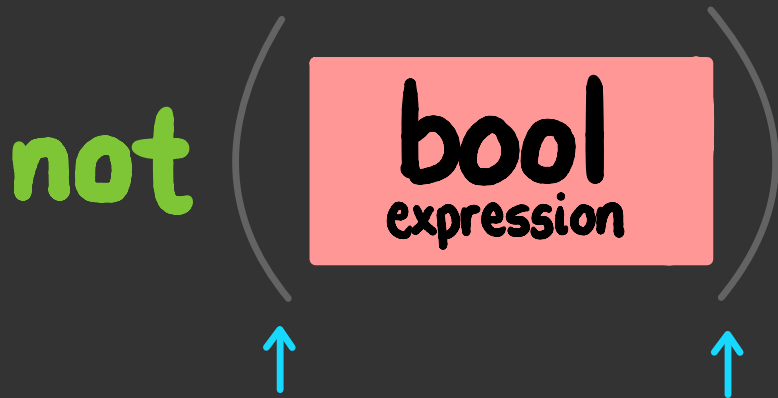
then the value of the overall
expression is False
(and vice versa)

```
In [1]: not(True)  
Out[1]: False
```

```
In [2]: not(4 > 3)  
Out[2]: False
```

```
In [3]: not(False)  
Out[3]: True
```

```
In [4]: not(3 > 4)  
Out[4]: True
```



the parens are
optional

```
In [1]: not True  
Out[1]: False
```

```
In [2]: not 4 > 3  
Out[2]: False
```

```
In [3]: not False  
Out[3]: True
```

```
In [4]: not 3 > 4  
Out[4]: True
```

```
In [1]: roll = 7
```

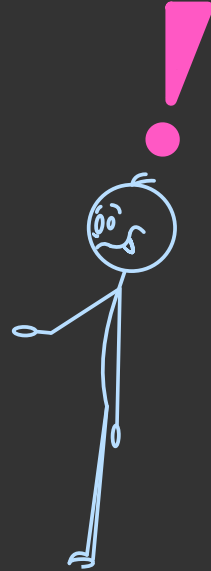
```
In [2]: roll == 7 or 11
```

```
Out[2]: True
```

```
In [3]: roll = 8
```

```
In [4]: roll == 7 or 11
```

```
Out[4]: 11
```



be careful of this

```
In [1]: roll = 7
```

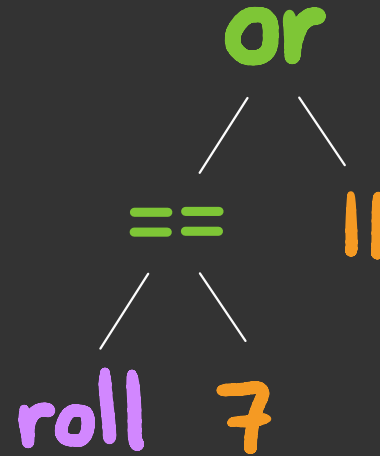
```
In [2]: roll == 7 or 11
```

```
Out[2]: True
```

```
In [3]: roll = 8
```

```
In [4]: roll == 7 or 11
```

```
Out[4]: 11
```



let's evaluate this expression tree

```
In [1]: roll = 7
```

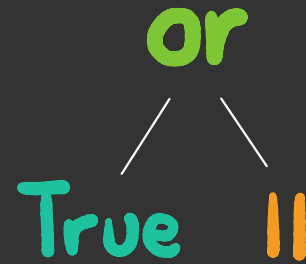
```
In [2]: roll == 7 or 11
```

```
Out[2]: True
```

```
In [3]: roll = 8
```

```
In [4]: roll == 7 or 11
```

```
Out[4]: 11
```



let's evaluate this expression tree

```
In [1]: roll = 7
```

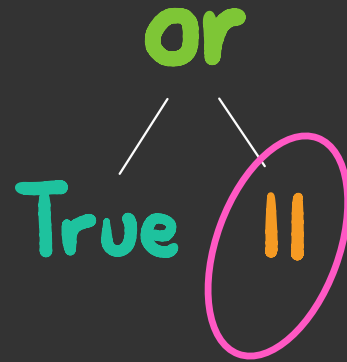
```
In [2]: roll == 7 or 11
```

```
Out[2]: True
```

```
In [3]: roll = 8
```

```
In [4]: roll == 7 or 11
```

```
Out[4]: 11
```



danger: **this** is not a bool expression

```
In [1]: roll = 7  
  
In [2]: roll == 7 or 11  
Out[2]: True  
  
In [3]: roll = 8  
  
In [4]: roll == 7 or 11  
Out[4]: 11
```



unexpected things may happen



```
In [1]: total = 7
```

```
In [2]: total == 7 or 11
```

```
Out[2]: True
```

```
In [3]: total = 8
```

```
In [4]: total == 7 or 11
```

```
Out[4]: 11
```



```
In [1]: total = 7
```

```
In [2]: (total == 7) or (total == 11)
```

```
Out[2]: True
```

```
In [3]: total = 8
```

```
In [4]: (total == 7) or (total == 11)
```

```
Out[4]: False
```

expressions
so far

type	example	value
int	7	7
float	3.14	3.14
str	"williams"	"williams"
bool	$3 > 4$	False